La création de formulaires

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I - Contexte **3**

II - Découverte des formulaires **4**

A. Comprendre l'utilisation et la fonction des formulaires 4

B. L'élément essentiel d'un formulaire 4

C. Création d'un premier formulaire 5

D. Exercice : Quiz 7

III - Les éléments **9**

A. Présentation des éléments 9

B. Créer une liste d'options déroulante 10

C. Créer des champs de saisie 10

D. Structurer et améliorer la sémantique d'un formulaire 10

E. Attributs et bonnes pratiques 10

F. Exercice : Quiz 12

IV - Essentiel **14**

V - Auto-évaluation **15**

A. Exercice 15

B. Test 15

Solutions des exercices **17**

Tous droits réservés à STUDI - Reproduction interdite

I Contexte

Durée : 1 h

Environnement de travail : Visual Studio Code

Contexte

Les formulaires HTML sont une partie essentielle de l'expérience utilisateur sur le web. Ils permettent aux utilisateurs d'interagir avec un site web, de soumettre des informations et d'effectuer des transactions. Bien que les formulaires puissent sembler simples à première vue, ils peuvent rapidement devenir complexes en fonction des besoins spécifiques d'un site web. C'est pourquoi il est important de comprendre les éléments, les attributs et les bonnes pratiques à suivre pour créer des formulaires efficaces, faciles à utiliser et sécurisés. En suivant les étapes que nous vous avons fournies dans ce cours, vous serez en mesure de créer des formulaires efficaces, optimisés et accessibles qui répondent aux normes d'accessibilité WCAG, qui vous aideront à interagir avec vos visiteurs et à recueillir les informations dont vous avez besoin pour améliorer votre site web.

Tous droits réservés à STUDI - Reproduction interdite

II Découverte des formulaires

A. Comprendre l'utilisation et la fonction des formulaires

Les formulaires permettent aux utilisateurs d'envoyer des données que nous pouvons ensuite manipuler et / ou stocker, que ce soit lors d'une inscription / connexion, d'un tri d'articles sur une page ou même d'un envoi de fichier. Dès qu'il s'agit de données utilisateur à envoyer, l'utilisation de formulaire est la méthode la plus courante.

Il est à noter qu'une certaine sécurité est nécessaire, mais de manière générale, c'est lors du traitement des données que l'on vérifie si les données reçues sont celles attendues. En effet, un utilisateur malveillant peut tenter de contourner les règles en effectuant une requête SQL, par exemple, mais pas d'inquiétude, cela fera l'objet d'un autre cours.

Ici, nous allons seulement aborder le langage HTML et nous assurer que les meilleurs éléments soient sélectionnés en fonction du contexte afin d'obtenir la sémantique la plus propre possible et ainsi obtenir un meilleur référencement et de meilleure performance.

Az Définition

La sémantique en HTML est l'utilisation correcte des balises pour structurer et décrire le contenu d'une page web de manière logique et compréhensible pour les navigateurs, les moteurs de recherche et les utilisateurs. Une sémantique appropriée peut aider à améliorer l'accessibilité, la maintenabilité et le référencement de la page web.

B. L'élément essentiel d'un formulaire

La création d'un formulaire s'effectue à l'aide de l'élément form. Celui-ci permet de regrouper tous les champs de saisie et boutons que nous verrons dans la seconde partie. L'élément `form` attend deux attributs : `action` et `method`. Ils sont d'une importance capitale.

L'attribut `action` permet de définir la page qui va recevoir et traiter les données. Par défaut, si on laisse le champ vide, la page où se trouve le container sera sélectionnée.

💬 Remarque

Dans le cas où vous renseignez un chemin comme valeur dans l'attribut `action`, une redirection se fera lors de l'envoi du formulaire sur la page en question.

L'attribut `method` permet de choisir comment les données seront envoyées. Il peut prendre les valeurs POST et GET.

1. La valeur GET indique que les données transiteront via l'URL,
2. La valeur POST, à l'inverse, indique que les données du formulaire transiteront via une transaction post HTTP.

Pour simplifier, la méthode GET utilise l'URL pour transmettre les données sous forme de paramètre, limitant ainsi la quantité de données. Il est également important de noter que les données sont visibles et modifiables par l'utilisateur. Quand il s'agit de données sensibles comme un mot de passe, il est préférable d'utiliser la méthode POST. Celle-ci permet de masquer le contenu envoyé et nous laisse une totale liberté sur le type et la taille des données.

Tous droits réservés à STUDI - Reproduction interdite

C. Création d'un premier formulaire

Place à la pratique ! Dans notre exemple, nous allons imaginer que notre formulaire est un formulaire d'inscription. Les trois éléments suivants devront être renseignés par l'utilisateur :

- Nom
- E-mail
- Mot de passe

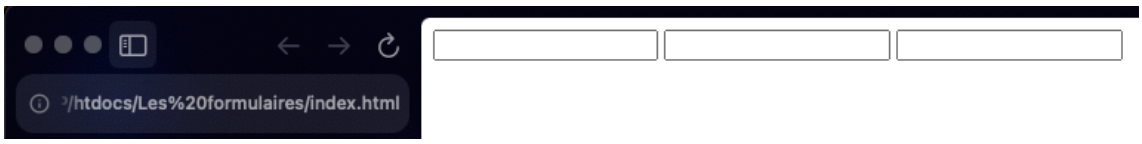
L'élément HTML qui nous permettra de récupérer ces informations est l'élément input. Cet élément nécessite obligatoirement l'attribut type, qui peut prendre de nombreuses valeurs. Le type permet de connaître le type de données que l'utilisateur va pouvoir renseigner. Par exemple, pour une date, le type sera « *date* », pour un e-mail, il sera « *email* ». Nous y reviendrons plus en détail plus tard.

> ⚠ Attention
>
> Les balises input sont des balises orphelines, c'est-à-dire qu'elles ne nécessitent pas de balise de fermeture.

Notez que chaque navigateur a une présentation différente pour chaque type. Si vous souhaitez créer un input générique, il existe des morceaux de code appelés « *normalize* » disponibles sur Internet, qui permettent d'écraser les styles des champs, des boutons, etc.

Notre formulaire aura donc la structure suivante :

```html
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>La création de formulaires</title>
</head>
<body>
    <form action="" method="POST">
        <input type="text">
        <input type="email">
        <input type="password">
    </form>
</body>
</html>
```



En seulement quelques lignes, nos trois champs ont été créés et le navigateur sait quels types de données chacun des trois champs attend. Si j'écris dans l'input qui a le type « *password* », le navigateur va automatiquement masquer mon texte à l'aide de points noirs.

Si votre navigateur reconnaît des similitudes avec des formulaires d'autres sites, il est possible qu'il vous propose une autocomplétion. Sympa, n'est-ce pas ?

Il nous reste un souci : l'accessibilité. Comment l'utilisateur peut-il savoir quelle information est attendue dans chaque champ ? Et vous l'avez sûrement remarqué, il manque également un bouton pour envoyer les données.

Tous droits réservés à STUDI - Reproduction interdite

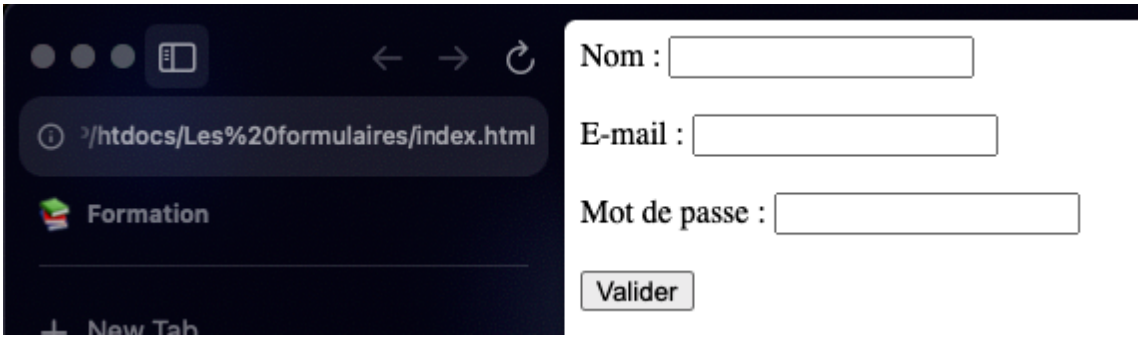
Un élément a été prévu à cet effet : le label. Comme son nom l'indique, il permet d'ajouter un libellé (un nom) à chaque champ de notre formulaire. Il est conseillé de lier les labels à leurs champs correspondants. Les labels deviennent cliquables et permettent de rediriger le focus du clavier sur leur input.

Pour effectuer la liaison, rien de plus simple : il suffit d'attribuer un id à chacun de nos inputs et d'ajouter l'attribut « *for* » qui aura comme valeur l'id de son input.

En ce qui concerne le bouton de soumission, il suffit de créer un nouvel input avec le type « *submit* », avec comme seule subtilité le fait que le label soit remplacé par l'attribut « *value* ». Dans notre exemple, la valeur de « *value* » sera « *Valider* ».

En somme, voici ce que nous obtenons :

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>La création de formulaires</title>
</head>
<body>
    <form action="" method="POST">
        <div>
            <label for="name">Nom :</label>
            <input id="name" type="text">
        </div><br/>
        <div>
            <label for="mail">E-mail :</label>
            <input id="mail" type="email">
        </div><br/>
        <div>
            <label for="pass">Mot de passe :</label>
            <input id="pass" type="password">
        </div><br/>
        <input type="submit" value="Valider">
    </form>
</body>
</html>
```



Remarque

Les balises div et br n'affectent pas le formulaire en soi, elles sont simplement utilisées pour séparer visuellement chaque champ.

Pour mieux comprendre, voici une démonstration concrète :

Tous droits réservés à STUDI - Reproduction interdite

D. Exercice : Quiz

[solution n°1 p. 17]

Question 1

Quel est l'élément qui permet de créer un formulaire ?

- Input
- Form
- Label

Question 2

Quel est l'attribut qui spécifie le type de champ d'entrée à afficher pour l'élément input ?

- Value
- Name
- Type

Question 3

Comment associer un label à un champ de formulaire ?

- À l'aide de l'attribut « `id` » du label
- À l'aide de l'attribut « `name` » du label
- À l'aide de l'attribut « `for` » du label

Question 4

Quel est l'attribut utilisé pour renseigner le chemin de la page où les données seront traitées lorsqu'un formulaire est soumis ?

- Action
- For
- Method

Question 5

Quel est l'attribut utilisé pour spécifier la méthode d'envoi de données à utiliser lors de la soumission d'un formulaire ?

Tous droits réservés à STUDI - Reproduction interdite

Method

Id

Action

Tous droits réservés à STUDI - Reproduction interdite

III Les éléments

A. Présentation des éléments

Dans la partie précédente, nous avons eu l'occasion de voir les éléments fondamentaux d'un formulaire, tels que l'élément « `form` » et l'élément « `input` ».

Il existe des éléments spécifiques pour répondre à chaque situation, que ce soit pour des champs de texte longs ou courts, des boutons à choix multiples ou à choix unique, etc.

Ci-dessous, vous trouverez la liste des éléments dédiés aux formulaires.

Désignation	Rôle
`form`	Définit un formulaire HTML qui peut être soumis à un serveur pour traitement.
`input`	Permet aux utilisateurs de saisir des données, telles que des champs de texte, des boutons radio, des cases à cocher, des champs de sélection, etc.
`button`	Crée un bouton cliquable.
`label`	Fournit une légende pour un élément de formulaire, aidant à associer le libellé à l'élément correspondant.
`textarea`	Permet aux utilisateurs de saisir de longs blocs de texte.
`option`	Définit les options disponibles dans un élément « *select* » ou « *datalist* ».
`optgroup`	Regroupe un ensemble d'options dans un élément « *select* » pour faciliter leur organisation.
`select`	Permet aux utilisateurs de sélectionner une ou plusieurs options à partir d'une liste déroulante.
`datalist`	Permet aux utilisateurs de saisir des données en les choisissant dans une liste de suggestions.
`legend`	Ajoute une légende à un élément fieldset.
`fieldset`	Regroupe un ensemble de champs liés avec une bordure et une légende optionnelle.

⚠ Attention

Bien que les éléments `button` et `input type button` puissent être utilisés pour créer des boutons cliquables, l'utilisation de l'élément `input type button` est généralement recommandée pour améliorer l'accessibilité et la flexibilité du code.

Tous droits réservés à STUDI - Reproduction interdite

B. Créer une liste d'options déroulante

L'élément « `select` » est utilisé en conjonction avec les éléments « `option` » pour chaque choix proposé.

Cette méthode permet à l'utilisateur de sélectionner un seul choix dans la liste, ce qui est souvent utilisé pour les questions qui proposent plusieurs choix, comme la sélection du département de naissance.

L'élément « `optgroup` » peut également être utilisé pour regrouper des options similaires, comme des pays par continent.

C. Créer des champs de saisie

Dans la majorité des cas, l'élément input en type `texte` est utilisé pour les données de type « *texte court* ».

Cependant, il est limité en termes de taille de texte, ce qui peut être un problème si l'on souhaite permettre aux utilisateurs de saisir des commentaires ou des descriptions par exemple.

Pour les textes plus longs, l'élément `textarea` est préférable, car il permet la saisie de textes plus longs.

> 💬 Remarque
>
> Le `textarea` donne également la possibilité à l'utilisateur de redimensionner la taille du champ de saisie (l'option peut être désactivée).

D. Structurer et améliorer la sémantique d'un formulaire

Il est possible d'utiliser l'élément `fieldset` qui permet de regrouper plusieurs champs dans une partie du formulaire pour l'organiser en parties. L'élément `legend` permet, comme son nom l'indique, d'ajouter une légende à une partie du formulaire pour expliquer son but.

Explorez comment les éléments et attributs sont utilisés dans un contexte réel.

E. Attributs et bonnes pratiques

Il est important de toujours avoir en tête qu'un utilisateur peut être mal intentionné et vouloir contourner votre formulaire. Étant donné que c'est à l'aide des formulaires que l'utilisateur communique avec le serveur et la base de données, il faut être vigilant dans votre structure, et cette dernière partie est là pour ça !

Mais pour s'en protéger, il faut connaître les types d'attaque possible. En effet, il en existe de nombreuses, mais parmi les plus plus répandues deux méthodes sont très utilisées. La première consiste à injecter un script JavaScript dans un champ de texte et la seconde à faire une requête SQL en essayant de contourner les accès administrateurs afin de pouvoir récupérer toutes les données sensibles du site.

Maintenant que vous avez connaissance des enjeux de la sécurité, particulièrement sur les formulaires, nous allons voir comment y pallier sur la partie HTML. Bien entendu, toute la partie traitement des informations côté serveur est importante.

Les bonnes pratiques

Il est nécessaire d'adopter des bonnes pratiques afin d'avoir la nomenclature et les performances les plus optimales possibles. Pour cela, HTML a prévu des attributs, notamment les plus courants qui sont `required`, `min` et `max`, qui permettront de forcer l'utilisateur à renseigner des informations et limiter le nombre de caractères.

Tous droits réservés à STUDI - Reproduction interdite

Imaginons que notre champ attende un âge jusqu'à 30 ans. La bonne méthode serait de limiter le nombre à maximum 30. Pour cela, rien de plus simple : utilisez l'attribut max avec comme valeur 30 pour ne pas pouvoir mettre au delà de 30. Dans un second temps, il faudra changer le type de l'input en number.

⚠ Attention

Toutes les vérifications faites par le navigateur peuvent cependant être neutralisées à l'aide de l'inspecteur notamment. La vérification au niveau du navigateur n'est donc pas suffisante et permet avant tout d'éviter les erreurs d'inattention et les attentes de chaque champ.

Liste des attributs

💬 Remarque

Une des bonnes pratiques qui aide l'expérience utilisateur est donc de renseigner un maximum d'attributs. Voici la liste des attributs à utiliser sans modération afin de guider au maximum l'utilisateur et éviter les étourderies.

Désignation	Rôle
`size`	Définit le nombre de caractères visibles dans un champ de texte.
`min`	Définit la valeur minimale pour un champ numérique.
`max`	Définit la valeur maximale pour un champ numérique.
`minlength`	Spécifie le nombre minimum de caractères requis dans un champ de texte.
`maxlength`	Spécifie le nombre maximum de caractères autorisés dans un champ de texte.
`autocomplete`	Indique si le formulaire doit être autocomplété par le navigateur.
`pattern`	Définit une expression régulière pour la validation des données entrées dans un champ de texte.
`required`	Indique que le champ est obligatoire.
`step`	Spécifie l'incrément pour les valeurs numériques dans un champ numérique.
`value`	Permet de définir une valeur par défaut.
`placeholder`	Permet d'afficher un texte demi-transparent si le champ est vide.

Tous droits réservés à STUDI - Reproduction interdite

F. Exercice : Quiz

[solution n°2 p. 18]

Question 1

Quel attribut permet de spécifier le type de données attendu dans un champ de saisie de texte ?

- `required`
- `pattern`
- `type`

Question 2

L'attribut `autocomplete` permet de préremplir automatiquement les champs de formulaire avec des données enregistrées précédemment.

- Vrai
- Faux

Question 3

Quel attribut HTML permet de définir une valeur par défaut pour un champ de formulaire ?

- `default`
- `value`
- `placeholder`

Question 4

L'attribut `maxlength` permet de limiter le nombre de caractères que l'utilisateur peut saisir dans un champ de formulaire.

- Vrai
- Faux

Question 5

Quel est le rôle de l'attribut `size` dans un formulaire ?

- Détermine la taille maximale des données pouvant être saisies dans un champ de texte
- Détermine le nombre de caractères visibles dans un champ de texte

Tous droits réservés à STUDI - Reproduction interdite

Détermine la largeur du champ de texte dans le formulaire

Question 6

L'élément `textarea` est utilisé pour créer un champ de saisie de texte long dans un formulaire HTML.

Vrai

Faux

Question 7

L'élément `label` est utilisé pour créer des boutons cliquables dans un formulaire.

Vrai

Faux

Question 8

Quel élément est utilisé pour grouper plusieurs champs dans un formulaire ?

`input type “group”`

Fieldset

`Legend`

Question 9

Quel élément HTML est utilisé pour ajouter une légende à une partie d'un formulaire ?

`Label`

`Input`

`Legend`

Tous droits réservés à STUDI - Reproduction interdite

IV Essentiel

Un formulaire permet à l'utilisateur de communiquer des informations et des fichiers vers un site web. Les données peuvent transiter de deux manières différentes, soit via l'URL (GET) soit par une transaction post HTTP (POST). L'utilisation de ces deux méthodes varie en fonction de la situation et de la sensibilité des données. Il existe plusieurs types d'éléments qui permettent de répondre à chaque situation, tels que des champs pour les dates, des zones de texte pour les longs textes, des cases à cocher, etc. De plus, des attributs peuvent être ajoutés pour guider l'utilisateur, augmenter la sécurité et éviter les erreurs d'inattention. Et pour finir, il est recommandé d'utiliser des labels pour indiquer à l'utilisateur quelles données sont attendues dans chaque champ et ainsi améliorer l'accessibilité.

Tous droits réservés à STUDI - Reproduction interdite

V Auto-évaluation

A. Exercice

Vous êtes chargé de créer un formulaire d'inscription pour un atelier de programmation. Le formulaire doit permettre aux participants de saisir leurs informations personnelles (nom, prénom, email) et de choisir un langage de programmation parmi une liste d'options.

Question 1

[solution n°3 p. 21]

Créez un formulaire simple permettant de répondre à la demande.

Question 2

[solution n°4 p. 22]

Pensez à rendre les champs obligatoires et ajouter une zone permettant aux personnes de laisser un commentaire.

B. Test

Exercice 1 : Quiz

[solution n°5 p. 22]

Question 1

Pour créer un formulaire, nous utilisons l'élément `fieldset`.

- Vrai
- Faux

Question 2

Parmi les choix suivants, quel est l'élément qui permet de créer une liste déroulante dans un formulaire ?

- `<input type="list">`
- `<select>`
- `<dropdown>`

Question 3

Les boutons radio sont créés en utilisant l'élément input avec l'attribut `type="radio"`.

- Vrai
- Faux

Question 4

Parmi les choix suivants, quel est l'élément qui permet de créer un champ de saisie de mot de passe dans un formulaire ?

Tous droits réservés à STUDI - Reproduction interdite

```
<input type="password">
```

```
<input type="text" mask="password">
```

```
<input type="secure">
```

Question 5

L'attribut `required` dans un élément `input` permet de rendre obligatoire la saisie dans ce champ.

Vrai

Faux

Tous droits réservés à STUDI - Reproduction interdite

Solutions des exercices

Solution n°1

[exercice p. 7]

Question 1

Quel est l'élément qui permet de créer un formulaire ?

- Input
- ✓ Form
- Label

En effet, l'élément form est l'élément principal pour créer un formulaire HTML. Il permet d'encapsuler tous les éléments du formulaire.

Question 2

Quel est l'attribut qui spécifie le type de champ d'entrée à afficher pour l'élément input ?

- Value
- Name
- ✓ Type

L'attribut « *type* » permet au navigateur de spécifier quel type de donnée est attendue.

Question 3

Comment associer un label à un champ de formulaire ?

- À l'aide de l'attribut « `id` » du label
- À l'aide de l'attribut « `name` » du label
- ✓ À l'aide de l'attribut « `for` » du label

Pour associer un label à un champ de formulaire, l'attribut « *for* » est nécessaire et il faut lui donner la valeur de l'ID de l'élément correspondant.

Tous droits réservés à STUDI - Reproduction interdite

Question 4

Quel est l'attribut utilisé pour renseigner le chemin de la page où les données seront traitées lorsqu'un formulaire est soumis ?

- ✓ Action
- For
- Method

En effet, l'attribut « *action* » permet de renseigner l'URL ou le chemin à appeler lorsqu'un formulaire est soumis.

Question 5

Quel est l'attribut utilisé pour spécifier la méthode d'envoi de données à utiliser lors de la soumission d'un formulaire ?

- ✓ Method
- Id
- Action

L'attribut « *`method`* » permet de choisir quelle méthode HTTP est à utiliser.

Solution n°2

[exercice p. 12]

Question 1

Quel attribut permet de spécifier le type de données attendu dans un champ de saisie de texte ?

- `required`
- `pattern`
- ✓ `type`

En effet, l'attribut `type` permet de définir le type de données attendu dans un champ de saisie de texte, comme du texte (`text`), une adresse e-mail (`email`), un numéro de téléphone (`tel`), etc.

Tous droits réservés à STUDI - Reproduction interdite

Question 2

L'attribut `autocomplete` permet de préremplir automatiquement les champs de formulaire avec des données enregistrées précédemment.

- ✓ Vrai
- Faux

L'attribut `autocomplete` permet de préremplir automatiquement les champs de formulaire avec des données enregistrées dans d'autres sites, comme les noms d'utilisateur, les adresses e-mail, les adresses postales, etc.

Question 3

Quel attribut HTML permet de définir une valeur par défaut pour un champ de formulaire ?

- `default`
- ✓ `value`
- `placeholder`

L'attribut `value` permet de définir une valeur par défaut qui sera préremplie lorsque la page sera chargée.

Question 4

L'attribut `maxlength` permet de limiter le nombre de caractères que l'utilisateur peut saisir dans un champ de formulaire.

- ✓ Vrai
- Faux

L'attribut `maxlength` permet de limiter le nombre de caractères que l'utilisateur peut saisir dans un champ de formulaire, en spécifiant le nombre maximal de caractères autorisés.

Question 5

Quel est le rôle de l'attribut `size` dans un formulaire ?

- Détermine la taille maximale des données pouvant être saisies dans un champ de texte

Tous droits réservés à STUDI - Reproduction interdite

- ✓ Détermine le nombre de caractères visibles dans un champ de texte
- Détermine la largeur du champ de texte dans le formulaire

L'attribut `size` est utilisé pour définir le nombre de caractères visibles dans un champ de texte. Il ne limite pas la taille des données pouvant être saisies, contrairement à l'attribut `maxlength`. La valeur de `size` détermine la largeur approximative du champ de texte, mais cette valeur peut être modifiée par le navigateur en fonction de la police de caractères utilisée.

Question 6

L'élément `textarea` est utilisé pour créer un champ de saisie de texte long dans un formulaire HTML.

- ✓ Vrai
- Faux

En effet, l'élément `textarea` est utilisé pour créer un champ de saisie de texte long dans un formulaire. Contrairement à l'élément `input` avec comme type `texte`, il n'y a pas de limite de caractères pour le texte saisi dans un `textarea`.

Question 7

L'élément `label` est utilisé pour créer des boutons cliquables dans un formulaire.

- Vrai
- ✓ Faux

L'élément `label` est utilisé pour associer un libellé à un champ de formulaire, tel qu'un champ de saisie ou une case à cocher. Pour créer des boutons cliquables dans un formulaire HTML, nous pouvons utiliser l'élément `button` ou l'attribut « *type* » de l'élément input avec les valeurs « *button* » ou « *submit* ».

Question 8

Quel élément est utilisé pour grouper plusieurs champs dans un formulaire ?

- `input type “group”`
- ✓ Fieldset

Tous droits réservés à STUDI - Reproduction interdite

Legend

En effet, le `fieldset` est utilisé pour grouper plusieurs champs dans un formulaire et les organiser en parties.

Question 9

Quel élément HTML est utilisé pour ajouter une légende à une partie d'un formulaire ?

Label

Input

✓ Legend

L'élément `legend` est utilisé pour ajouter une légende à une partie d'un formulaire. Il doit être inclus dans un élément `fieldset` pour être associé à la partie concernée. La légende permet d'expliquer le but ou le contenu de la partie de formulaire concernée.

[exercice p. 15]

Solution n°3

```html
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Formulaire avec champs requis et textarea</title>
</head>
<body>
    <form>
        <label for="nom">Nom :</label>
        <input id="nom" type="text">
        <br>
        <label for="prenom">Prénom :</label>
        <input id="prenom" type="text">
        <br>
        <label for="email">E-mail :</label>
        <input id="email" type="email">
        <br>
       <label for="langage">Langage :</label>
                <select id="langage">
                    <option value="">Sélectionnez un langage</option>
                    <option value="python">Python</option>
                    <option value="java">Java</option>
                    <option value="javascript">JavaScript</option>
                    <option value="php">PHP</option>
                    <option value="ruby">Ruby</option>
                </select>
        <input type="submit" value="Valider">
    </form>
```

Tous droits réservés à STUDI - Reproduction interdite

```
</body>
</html>
```

[exercice p. 15]

Solution n°4

```
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Formulaire avec champs requis et textarea</title>
</head>
<body>
    <form>
        <label for="nom">Nom :</label>
        <input id="nom" type="text" required>
        <br>
        <label for="prenom">Prénom :</label>
        <input id="prenom" type="text" required>
        <br>
        <label for="email">E-mail :</label>
        <input id="email" type="email" required>
        <br>
        <label for="langage">Langage :</label>
        <select id="langage" required>
            <option value="">Sélectionnez un langage</option>
            <option value="python">Python</option>
            <option value="java">Java</option>
            <option value="javascript">JavaScript</option>
            <option value="php">PHP</option>
            <option value="ruby">Ruby</option>
        </select>
        <br>
        <label for="commentaire">Commentaire :</label>
        <textarea id="commentaire" required></textarea>
        <br>
        <input type="submit" value="Valider">
    </form>
</body>
</html>
```

Solution n°5

[exercice p. 15]

Question 1

Pour créer un formulaire, nous utilisons l'élément `fieldset`.

Vrai

✓ Faux

L'élément `fieldset` permet de grouper des champs dans un formulaire, la balise `form` est la balise principale qui permet de créer un formulaire. Elle est utilisée pour définir le début et la fin du formulaire.

Tous droits réservés à STUDI - Reproduction interdite

Question 2

Parmi les choix suivants, quel est l'élément qui permet de créer une liste déroulante dans un formulaire ?

`<input type="list">`

✓ `<select>`

`<dropdown>`

En effet, l'élément select est utilisé pour créer une liste déroulante dans un formulaire.

Question 3

Les boutons radio sont créés en utilisant l'élément input avec l'attribut `type="radio"`.

✓ Vrai

Faux

Les boutons radio sont créés en utilisant l'élément input avec l'attribut `type="radio"`. Nous utilisons des boutons radio pour permettre à l'utilisateur de choisir une option parmi plusieurs (le choix est unique).

Question 4

Parmi les choix suivants, quel est l'élément qui permet de créer un champ de saisie de mot de passe dans un formulaire ?

✓ `<input type="password">`

`<input type="text" mask="password">`

`<input type="secure">`

En effet, pour créer un champ de saisie de mot de passe dans un formulaire, nous utilisons l'élément `<input>` avec l'attribut `type="password"`.

Question 5

L'attribut `required` dans un élément `input` permet de rendre obligatoire la saisie dans ce champ.

Tous droits réservés à STUDI - Reproduction interdite

- ✓ Vrai
- Faux

L'attribut `required` dans un élément `input` permet de rendre obligatoire la saisie dans ce champ. Si l'utilisateur ne remplit pas le champ, il ne pourra pas soumettre le formulaire.

Tous droits réservés à STUDI - Reproduction interdite